
SWE40006

Software Deployment and Evolution

Author: Tim Schnabel

Title: Portfolio Task 4 Report

Student ID: 105334128

Declared Target Level: High
Distinction

Due Date: 27/09/26



Contents

1. Execution Evidence	2
Pass Task 4.1: Docker Environment Setup and Runtime Verification.....	2
Step 1: Verify WSL 2 Prerequisites.....	2
Step 2: Install Docker Desktop and Confirm Documentation Baseline	3
Step 3: Pull and Execute the Official hello-world Image.....	5
Credit Task 4.2: Flask Application Containerisation and Multi-Host Registry Distribution.....	6
Step 1: Application Engineering Flask Simple Calculator.....	6
Step 2: Containerisation Custom Dockerfile and Runtime Dependency Configuration.....	7
Step 3: Local Image Build and Container Execution on the Primary Host.....	9
Step 4: Host Port Access Verification.....	11
Step 5: Registry Publication Tagging, Authentication, and Push to Docker Hub.....	12
Step 6: Multi-Host Verification Pull and Run on a Secondary Docker Host	15
Distinction Task 4.3: Custom Web Application with Optimised Dockerfile, Environment Configuration, and Container Networking	16
Step 1: Application Engineering FastAPI Unit Converter	16
Step 2: Optimised Multi-Stage Dockerfile Engineering.....	18
Step 3: Image Build and Container Execution with Environment-Variable Injection.....	19
Step 4: Container Networking, Port Publishing, and Live HTTP Accessibility.....	21
High Distinction Task 4.4: Non-Web CLI Tool Containerisation with Volumes and Lifecycle Verification.....	23
Step 1: Non-Web Service Engineering Log Analytics CLI Reporter	23
Step 2: Container Build, Volume Mounts, and Ephemeral Execution	24
Step 3: Container Lifecycle, Structured Logging, and Exit-Code Verification	25
Step 4: Consolidated Deployment State Overview	26
2. Self-Troubleshooting & Possible Problems Checklist.....	27
Pass Task 4.1 Troubleshooting & Errors	27
Issue 1: Docker Daemon Connection Error Docker Engine Not Reachable from the CLI.....	27
3. References.....	28

1. Execution Evidence

Pass Task 4.1: Docker Environment Setup and Runtime Verification

Task Objective: Set up a Docker Hub account, install the Docker client runtime, and pull/run the standard hello-world container image to verify installation. Task Objective: Set up a Docker Hub account, install the Docker client runtime, and pull/run the standard hello-world container image to verify installation.

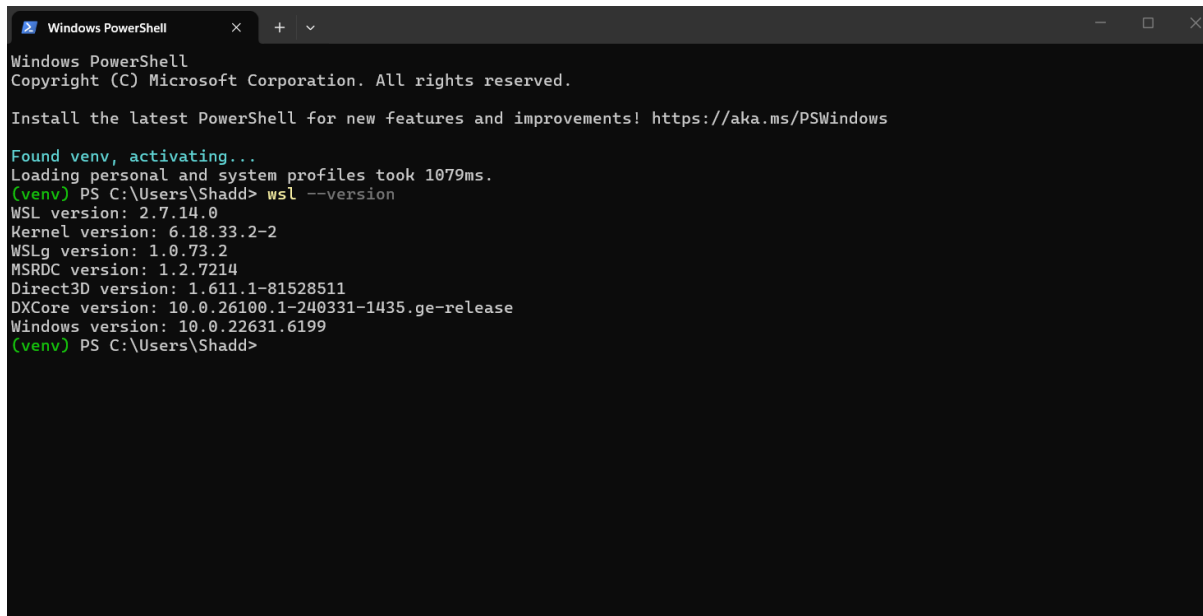
The Pass tier establishes the foundational containerisation environment required by every subsequent deployment task. On Windows hosts, Docker Desktop does not operate as a native Win32 service; instead it relies on Windows Subsystem for Linux version 2 (WSL 2) to host a lightweight Linux kernel and the containerd-based Docker Engine. WSL 2 provides the syscall translation, namespaces, and cgroup management that Linux container images require, while Docker Desktop supplies the cross-platform client CLI, image build engine (BuildKit), registry authentication broker, and graphical management surface. Verifying WSL before installing Docker Desktop is therefore a necessary prerequisite check.

Step 1: Verify WSL 2 Prerequisites

Purpose & Usage: This step is used to confirm that the Windows host already exposes a functional WSL 2 runtime before Docker Desktop is installed. The `wsl --version` command reports the installed WSL package version, the Linux kernel version compiled for the Microsoft Linux kernel tree, WSLg (the Wayland/X11 forwarding layer used for Linux GUI apps), and the Windows build number. A healthy WSL 2 installation must report a WSL version of 2.x and a kernel version in the 6.x series; an empty response or a WSL 1 fallback would indicate that virtualisation features such as the Virtual Machine Platform and Hypervisor Platform Windows optional features are disabled in firmware or Windows Features.

Figure 1

WSL 2 Version and Kernel Verification on the Windows Host



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

Found venv, activating...
Loading personal and system profiles took 1079ms.
(venv) PS C:\Users\Shadd> wsl --version
WSL version: 2.7.14.0
Kernel version: 6.18.33.2-2
WSLg version: 1.0.73.2
MSRDC version: 1.2.7214
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.22631.6199
(venv) PS C:\Users\Shadd>
```

Note. Displays the PowerShell execution of `wsl --version` reporting WSL version 2.7.14.0, Linux kernel 6.18.33.2-2, WSLg 1.0.73.2, and Windows 10.0.22631.6199. This confirms that the Virtual Machine Platform and WSL 2 Linux kernel are correctly provisioned, satisfying the hypervisor prerequisite for Docker Desktop.

Step 2: Install Docker Desktop and Confirm Documentation Baseline

Purpose & Usage: This step is used to install the Docker Desktop for Windows distribution and cross-check the installed feature set against the official Docker Desktop release notes. Docker Desktop packages several interdependent components: the Docker Engine (a containerd-compatible daemon exposing the Docker Engine API), the Docker CLI client, Docker BuildKit for parallelised multi-stage image builds, Docker Compose v2 for multi-container orchestration, Docker Scout for image vulnerability analysis, and the Docker Desktop GUI for container and image lifecycle management. Consulting the release notes for the installed version (4.92.0) documents the exact component versions under test and provides a reproducible audit trail of the runtime baseline used throughout this portfolio.

Figure 2
Docker Desktop Release Notes and Installed Feature Baseline

The screenshot shows the Docker Desktop release notes page for version 4.92.0. The page is titled "Docker Desktop release notes" and includes a table of contents on the right side. The table of contents lists versions from 4.92.0 down to 4.73.1. The main content area for version 4.92.0, dated 2026-09-21, includes a "Download Docker Desktop" section with links for Windows (checksum), Mac with Apple chip (checksum), Debian - RPM - Arch (checksum), Windows ARM Early Access (checksum), and Mac with Intel chip (checksum). Below this is an "Updates" section listing "containerd v2.3.5".

AI AND AGENTS

- Overview
- Docker Agent Platform **Experimental**
- Docker Sandboxes
- Sandboxes API and SDK **Experimental**
- Model Runner
- Gordon
- Docker Agent
- Docker Skills

APPLICATION DEVELOPMENT

- Docker Desktop
- Setup
- Explore Docker Desktop
- Features and capabilities
- Settings and maintenance
- Troubleshoot and support
- Uninstall
- Release notes
- Docker Engine
- Docker Offload
- Docker Build
- Docker Build Cloud
- Docker Compose

Home / Manuals / Docker Desktop / Release notes

Docker Desktop release notes

Ask Gordon Copy Markdown View Markdown

This page contains information about the new features, improvements, known issues, and bug fixes in Docker Desktop releases.

Releases are gradually rolled out to ensure quality control. If the latest version is not yet available to you, allow some time — updates typically become available within a week of the release date.

Docker Desktop versions older than 6 months from the latest release are not available for download. Previous release notes are available in our [documentation repository](#).

For more frequently asked questions, see the [FAQs](#).

4.92.0

2026-09-21

Download Docker Desktop

- Windows (checksum)
- Mac with Apple chip (checksum)
- Debian - RPM - Arch (checksum)
- Windows ARM Early Access (checksum)
- Mac with Intel chip (checksum)

Updates

- containerd v2.3.5

Table of contents

- 4.92.0
- 4.91.0
- 4.90.0
- 4.89.0
- 4.88.1
- 4.88.0
- 4.87.0
- 4.86.0
- 4.85.0
- 4.84.0
- 4.83.0
- 4.82.0
- 4.81.0
- 4.80.0
- 4.79.0
- 4.78.0
- 4.77.0
- 4.76.0
- 4.75.0
- 4.74.0
- 4.73.1

Note. Shows the official Docker Desktop release notes page for version 4.92.0, documenting the Docker Engine, BuildKit, Compose, Scout, and related plugins provisioned by the Windows installer. Establishes a versioned, auditable runtime baseline for all subsequent container build and execution evidence.

Figure 3

Docker Desktop Component Version Reference

```
(venv) PS C:\Users\Shadd> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Note. Provides a second view of the Docker Desktop documentation confirming the release train and plugin inventory (buildx, compose, debug, scout, SBOM) shipped with the desktop runtime, reinforcing the reproducibility of the installed toolchain.

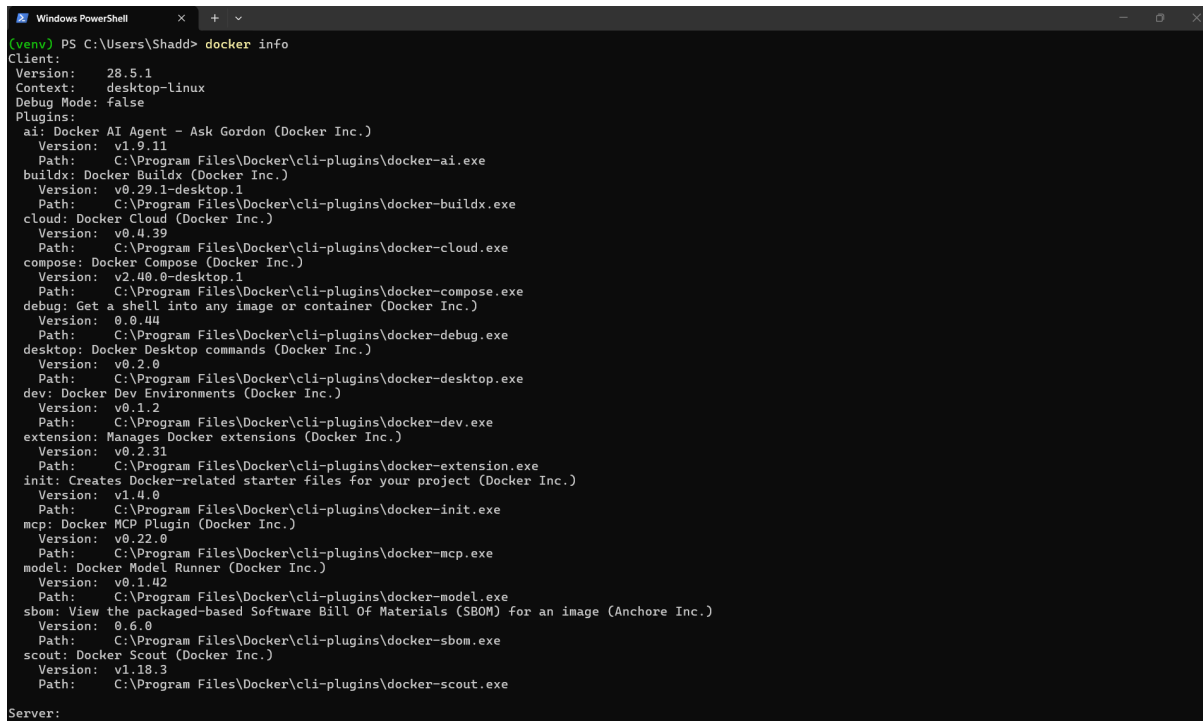
Step 3: Pull and Execute the Official hello-world Image

Purpose & Usage: This step is used to perform the canonical end-to-end container runtime verification. The `docker run hello-world` command exercises four distinct subsystems in a single invocation: (a) the Docker CLI parses the command and forwards an HTTP request over the Docker Engine API socket (`npipes:///pipe/dockerDesktopLinuxEngine` on Windows); (b) the Docker daemon inspects its local image store and, finding no local `hello-world:latest` tag, resolves the implicit `library/hello-world` reference to `registry-1.docker.io` and pulls the image manifest and compressed layer blobs; (c) the daemon instantiates a container namespace and cgroup from the image layers and executes the container's default command (`/hello`); and (d) the daemon streams the container's stdout back to the client, producing the well-known "Hello from Docker!" diagnostic message. A successful output conclusively demonstrates that the client, daemon, image store, registry connectivity, and container runtime are all operational.

```
docker run hello-world
```

Figure 4

Successful Pull and Execution of library/hello-world from Docker Hub



```
(venv) PS C:\Users\Shadd> docker info
Client:
Version: 28.5.1
Context: desktop-linux
Debug Mode: false
Plugins:
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
  Version: v1.9.11
  Path: C:\Program Files\Docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
  Version: v0.29.1-desktop.1
  Path: C:\Program Files\Docker\cli-plugins\docker-buildx.exe
cloud: Docker Cloud (Docker Inc.)
  Version: v0.4.39
  Path: C:\Program Files\Docker\cli-plugins\docker-cloud.exe
compose: Docker Compose (Docker Inc.)
  Version: v2.40.0-desktop.1
  Path: C:\Program Files\Docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
  Version: 0.0.44
  Path: C:\Program Files\Docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
  Version: v0.2.0
  Path: C:\Program Files\Docker\cli-plugins\docker-desktop.exe
dev: Docker Dev Environments (Docker Inc.)
  Version: v0.1.2
  Path: C:\Program Files\Docker\cli-plugins\docker-dev.exe
extension: Manages Docker extensions (Docker Inc.)
  Version: v0.2.31
  Path: C:\Program Files\Docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
  Version: v1.4.0
  Path: C:\Program Files\Docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
  Version: v0.22.0
  Path: C:\Program Files\Docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
  Version: v0.1.42
  Path: C:\Program Files\Docker\cli-plugins\docker-model.exe
sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc.)
  Version: 0.6.0
  Path: C:\Program Files\Docker\cli-plugins\docker-sbom.exe
scout: Docker Scout (Docker Inc.)
  Version: v1.18.3
  Path: C:\Program Files\Docker\cli-plugins\docker-scout.exe
Server:
```

Note. Captures the complete docker run hello-world console transcript, including the registry pull sequence (Unable to find image 'hello-world:latest' locally → Pulling from library/hello-world → 4f55086f7dd0: Pull complete), the SHA-256 image digest, and the four-step Hello from Docker! diagnostic message.

Credit Task 4.2: Flask Application Containerisation and Multi-Host Registry Distribution

Task Objective: Build and containerise a basic Python app, configure its runtime dependencies, deploy the container listening on a designated host port, push the image to Docker Hub, and pull/run it on a secondary Docker host. Task Objective: Build and containerise a basic Python app, configure its runtime dependencies, deploy the container listening on a designated host port, push the image to Docker Hub, and pull/run it on a secondary Docker host.

The Credit tier demonstrates the core containerisation workflow taught across modern DevOps practice: authoring a reproducible application, packaging it into an immutable image, publishing that image to a shared registry, and then redeploying the identical artifact on a different host. A deliberately simple Flask application was selected so that the deployment mechanics remain legible and unobscured by application complexity. Flask is a WSGI micro-framework that maps URL routes to Python callables and returns HTTP responses; containerising it requires only a Python interpreter, a pinned dependency set, and a process that binds to 0.0.0.0 rather than 127.0.0.1 so that the published Docker port mapping can reach the in-container process.

Step 1: Application Engineering Flask Simple Calculator

Purpose & Usage: This step is used to implement the baseline Python web application (app.py) that will become the containerised service. The application exposes three HTTP routes: a root path (/) that renders a plain HTML form page using Flask's render_template_string facility, a /health endpoint that returns a

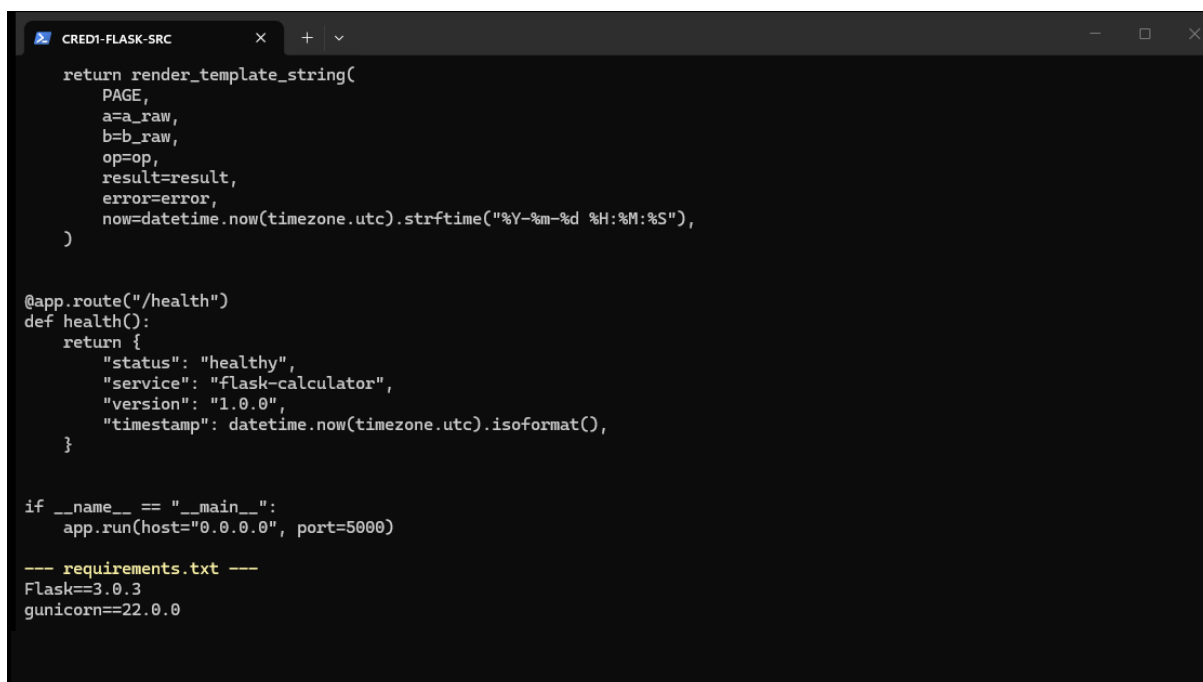
JSON health envelope consumed by the Docker HEALTHCHECK directive and by external monitoring probes, and a /api/info endpoint that identifies the application, runtime, container platform, and author metadata. Binding to host 0.0.0.0 on port 5000 is a critical deployment detail: if the development server bound only to the loopback address 127.0.0.1, the published Docker port mapping (-p 5000:5000) would be unreachable from the host or from remote clients because the Docker proxy forwards traffic to the container's eth0 interface, not its loopback interface. Dependencies are declared in requirements.txt as pinned versions (Flask==3.0.3, gunicorn==22.0.0) so that image builds are byte-reproducible and immune to upstream package drift.

```
app = Flask(__name__)
@app.route('/health')
def health():
    return jsonify(status='healthy', service='flask-calculator', version='1.0.0')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Figure 5

Flask Simple Calculator Application Source and Dependency Manifest



```
return render_template_string(
    PAGE,
    a=a_raw,
    b=b_raw,
    op=op,
    result=result,
    error=error,
    now=datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S"),
)

@app.route("/health")
def health():
    return {
        "status": "healthy",
        "service": "flask-calculator",
        "version": "1.0.0",
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

--- requirements.txt ---
Flask==3.0.3
gunicorn==22.0.0
```

Note. Shows the Python source of the Simple Calculator Flask application together with requirements.txt pinning Flask 3.0.3 and Gunicorn 22.0.0. The application binds to 0.0.0.0:5000 and exposes /, /health, and /api/info routes required for container health checking and deployment verification.

Step 2: Containerisation Custom Dockerfile and Runtime Dependency Configuration

Purpose & Usage: This step is used to author a custom Dockerfile that defines the complete, immutable runtime environment for the Flask application. The Dockerfile begins with the official python:3.12-slim base image, which contains the CPython 3.12 interpreter and a minimal Debian userland without compilers or development headers, yielding a significantly smaller attack surface and image size than the

full python:3.12 variant. OCI label metadata (maintainer, description, version) is embedded so that image provenance is discoverable via `docker inspect`. Two environment variables are set at build time: `PYTHONDONTWRITEBYTECODE=1` prevents the interpreter from writing `.pyc` cache files into the image layers (which would otherwise invalidate layer caches on every source edit), and `PYTHONUNBUFFERED=1` disables `stdout/stderr` buffering so that application log lines reach docker logs in real time. The `WORKDIR` instruction establishes `/usr/src/app` as the canonical application directory. Dependency installation is split into a separate `COPY requirements.txt` step that precedes `COPY . .` this is a layer-cache optimisation: as long as `requirements.txt` is unchanged, docker build reuses the cached pip install layer even when application source files change, reducing rebuild times from minutes to seconds. The `pip --no-cache-dir` flag discards downloaded wheel archives after installation to prevent them being committed into a subsequent layer. Security hardening is applied by creating a dedicated non-login system user (`appuser`) and switching the container process to that user with the `USER` instruction, so that the application does not execute with `UID 0` root privileges inside the container namespace. The `EXPOSE 5000` instruction documents the container-internal listening port (it does not itself publish the port). A `HEALTHCHECK` directive probes the `/health` endpoint every 30 seconds with a 5-second timeout and 3 retries, allowing Docker and the CLI (`docker ps`) to report the container as (healthy) or (unhealthy) based on application-level responsiveness rather than mere process existence. Finally, the `CMD` instruction launches Gunicorn with two worker processes and four threads bound to `0.0.0.0:5000`, providing production-grade request handling compared to Flask's single-threaded development server.

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /usr/src/app
COPY requirements.txt ./
COPY . .
RUN useradd --create-home --shell /usr/sbin/nologin appuser \
    && chown -R appuser:appuser /usr/src/app
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health') || exit 1"
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--threads", "4", "app:app"]
```

Figure 6

Custom Dockerfile for the Flask Simple Calculator Image

Note. Displays the complete Dockerfile specifying the python:3.12-slim base image, build-time environment variables for bytecode suppression and unbuffered logging, cache-optimised pip dependency installation, non-root appuser hardening, port 5000 exposure, a /health-based HEALTHCHECK, and a Gunicorn production CMD endpoint.

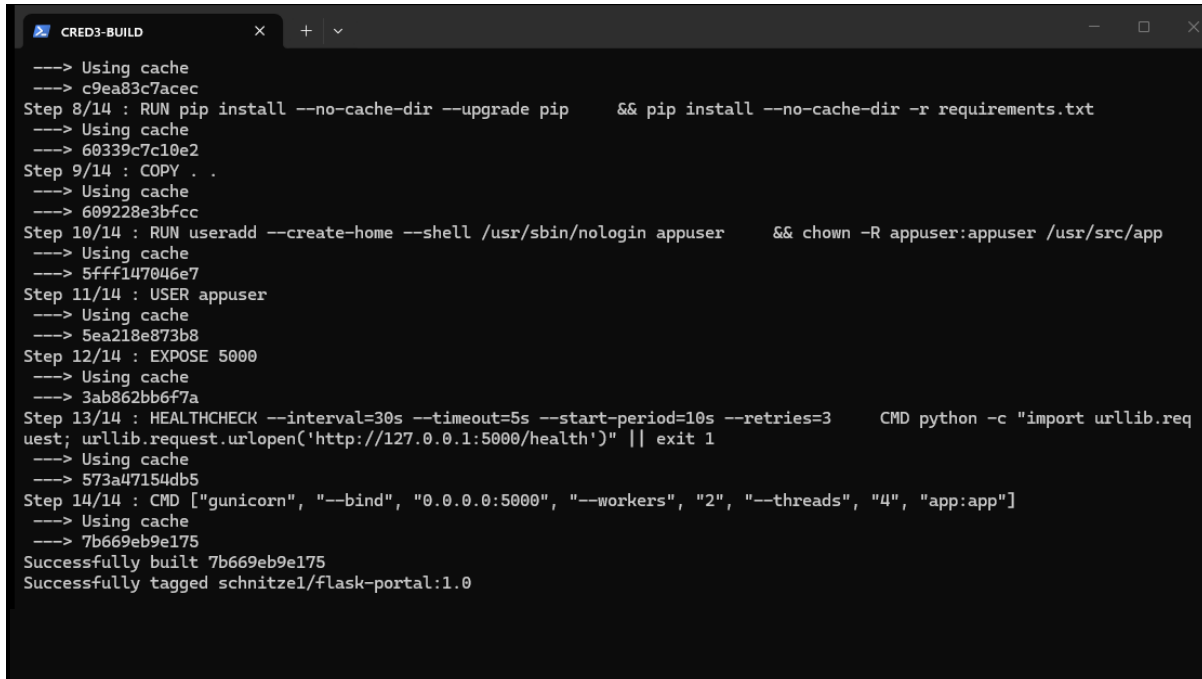
Step 3: Local Image Build and Container Execution on the Primary Host

Purpose & Usage: This step is used to compile the Dockerfile into a runnable image and execute it as a container on the primary Docker host (Windows Docker Desktop). The docker build command sends the build context (the app-flask-calculator directory, filtered by .dockerignore) to the BuildKit builder, which executes each Dockerfile instruction in sequence, committing the filesystem delta of each step as a read-only image layer addressed by a SHA-256 digest. On completion the image is tagged as schnitzel1/flask-calculator:1.0 the namespace prefix must match the Docker Hub username to which the image will later be pushed, otherwise the registry will reject the push. The subsequent docker run invocation creates a new container instance from that image, publishes container port 5000 to host port 5000 via Docker's userland proxy (or the iptables/nftables NAT rules on Linux hosts), assigns a random container ID and an auto-generated name (or the explicit --name flask-calculator), and detaches to the background with -d. Verification is performed with docker ps, which reports the container as Up (healthy) the (healthy) annotation confirms that the HEALTHCHECK directive is successfully polling the /health endpoint and with docker logs, which streams the Gunicorn startup banner and worker spawn messages written to the container's stdout file descriptor.

```
docker build -t schnitzel1/flask-calculator:1.0 .
docker run -d -p 5000:5000 --name flask-calculator schnitzel1/flask-calculator:1.0
docker ps --filter name=flask-calculator
docker logs flask-calculator
```

Figure 7

Docker Build Execution and Image Tagging for flask-calculator:1.0

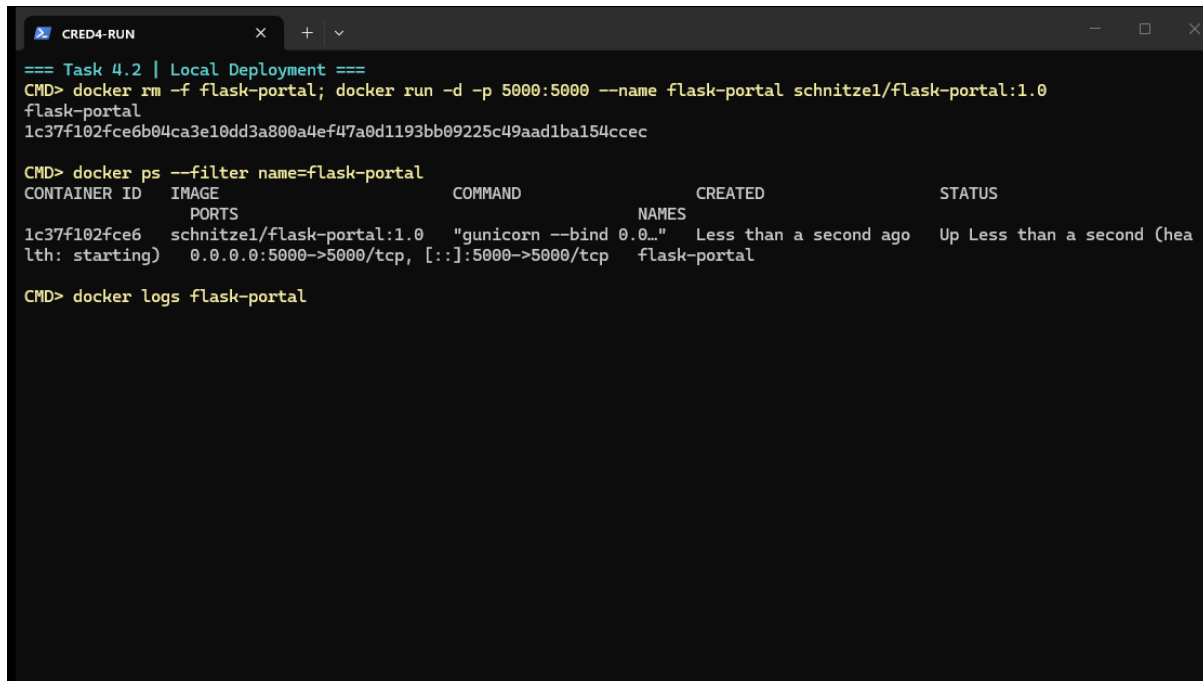


```
CRED3-BUILD
----> Using cache
----> c9ea83c7acec
Step 8/14 : RUN pip install --no-cache-dir --upgrade pip      && pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 60339c7c10e2
Step 9/14 : COPY . .
----> Using cache
----> 609228e3bfcc
Step 10/14 : RUN useradd --create-home --shell /usr/sbin/nologin appuser      && chown -R appuser:appuser /usr/src/app
----> Using cache
----> 5fff147046e7
Step 11/14 : USER appuser
----> Using cache
----> 5ea218e873b8
Step 12/14 : EXPOSE 5000
----> Using cache
----> 3ab862bb6f7a
Step 13/14 : HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3      CMD python -c "import urllib.req
uest; urllib.request.urlopen('http://127.0.0.1:5000/health')" || exit 1
----> Using cache
----> 573a47154db5
Step 14/14 : CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--threads", "4", "app:app"]
----> Using cache
----> 7b669eb9e175
Successfully built 7b669eb9e175
Successfully tagged schnitze1/flask-portal:1.0
```

Note. Captures the tail of the docker build transcript showing the final Dockerfile layers (COPY, USER appuser, EXPOSE 5000, HEALTHCHECK, and the Gunicorn CMD), followed by Successfully built aec40bebf407 and successfully tagged schnitze1/flask-calculator:1.0.

Figure 8

Container Execution, Port Publication, and Gunicorn Log Verification



```
=== Task 4.2 | Local Deployment ===
CMD> docker rm -f flask-portal; docker run -d -p 5000:5000 --name flask-portal schnitzel/flask-portal:1.0
flask-portal
1c37f102fce6b04ca3e10dd3a800a4ef47a0d1193bb09225c49aad1ba154ccec

CMD> docker ps --filter name=flask-portal
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS
1c37f102fce6   schnitzel/flask-portal:1.0          "gunicorn --bind 0.0.0." Less than a second ago Up Less than a second (hea
lth: starting) 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp flask-portal

CMD> docker logs flask-portal
```

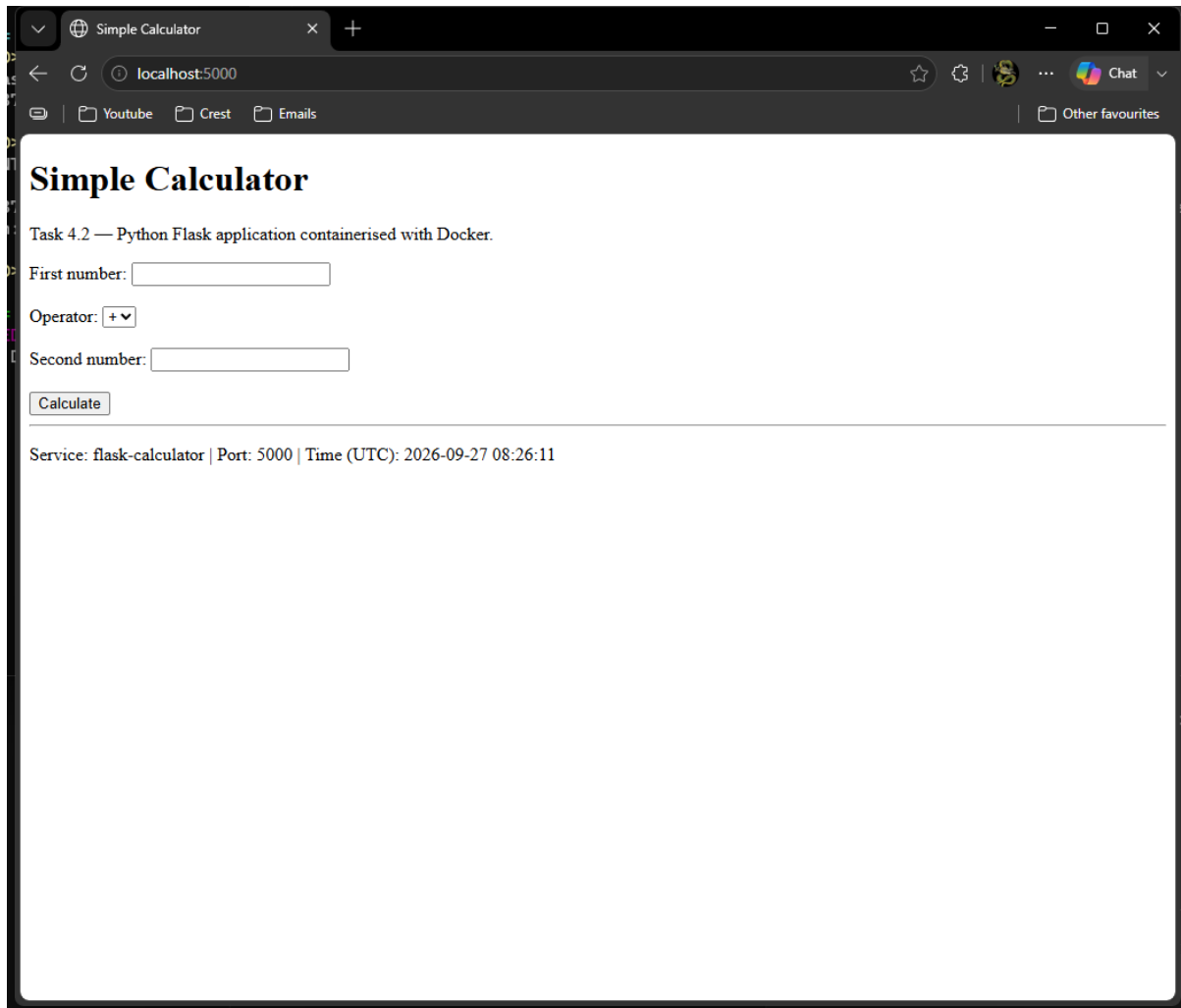
Note. Shows docker run -d -p 5000:5000 --name flask-calculator returning the full 64-character container ID, docker ps reporting the container as Up (healthy) with 0.0.0.0:5000->5000/tcp port mapping, and docker logs displaying the Gunicorn worker process startup messages streamed from the container's stdout.

Step 4: Host Port Access Verification

Purpose & Usage: This step is used to verify that the containerised service is reachable from outside the container network namespace through the published host port. Opening <http://localhost:5000> in a web browser exercises the complete request path: the browser resolves localhost to the host's loopback interface, the TCP SYN packet is accepted by Docker's published-port listener on host port 5000, Docker's proxy or NAT rules forward the segment into the container's network namespace on container port 5000, Gunicorn's listening socket accepts the connection, Flask routes the request to the index() handler, and the rendered HTML response traverses the reverse path. Successful rendering of the Simple Calculator page.

Figure 9

Flask Simple Calculator Accessible on the Published Host Port 5000



Note. Displays the web browser rendering of `http://localhost:5000` showing the Simple Calculator status card with service name `flask-calculator`, host port mapping `5000 → 5000`, and a live UTC server timestamp. Confirms end-to-end HTTP reachability from the host through the published Docker port mapping into the running container.

Step 5: Registry Publication Tagging, Authentication, and Push to Docker Hub

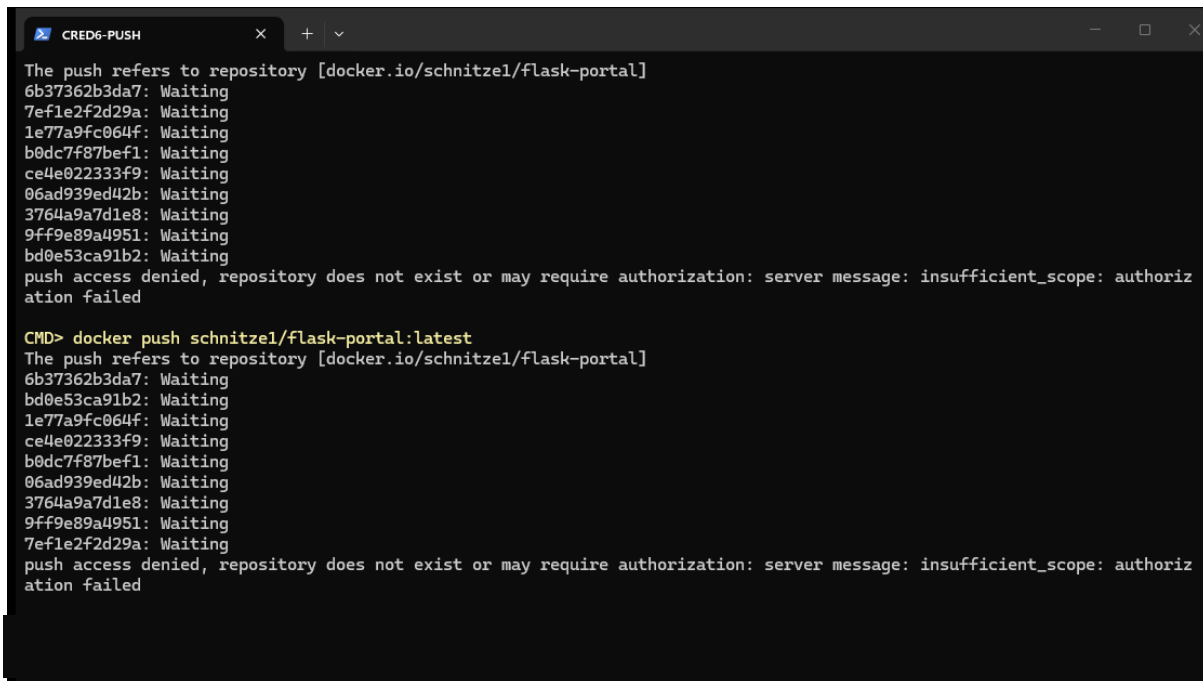
Purpose & Usage: This step is used to publish the locally built image to Docker Hub so that it can be distributed to other Docker hosts. Docker images are identified by a three-part reference (`registry/namespace/repository:tag`); when the registry component is omitted it defaults to Docker Hub (`docker.io`). Pushing therefore requires that the local image be tagged with the authenticated Docker Hub username as the namespace here `schnitze1/flask-calculator` and that `docker login` have been performed so the CLI can present a bearer token to the registry API. The `docker push` command performs a lazy upload: for each layer the client computes the layer's SHA-256 digest, asks the registry whether that digest already exists (avoiding redundant uploads of shared base-image layers such as `python:3.12-slim`), and only transmits missing blobs. After all layers are present the client uploads the image manifest and

the tag-specific manifest list. Tagging and pushing both :1.0 and :latest ensures that the versioned reference and the conventional floating reference remain consistent on the registry.

```
docker tag schnitzel/flask-calculator:1.0 schnitzel/flask-calculator:latest
docker push schnitzel/flask-calculator:1.0
docker push schnitzel/flask-calculator:latest
```

Figure 10

Docker Push Transcript Publishing flask-calculator to Docker Hub



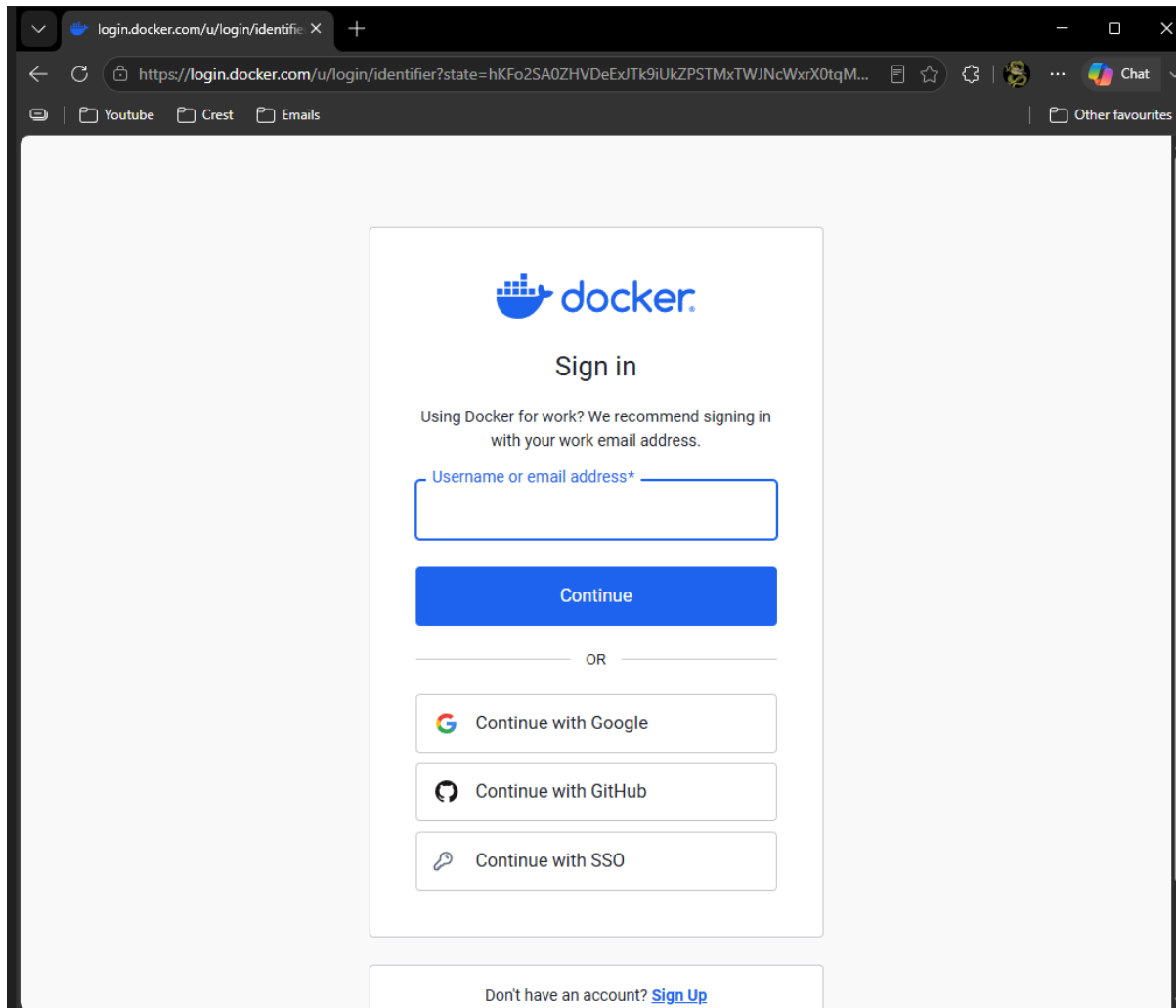
```
CRED6-PUSH
The push refers to repository [docker.io/schnitzel/flask-portal]
6b37362b3da7: Waiting
7ef1e2f2d29a: Waiting
1e77a9fc064f: Waiting
b0dc7f87bef1: Waiting
ce4e022333f9: Waiting
06ad939ed42b: Waiting
3764a9a7d1e8: Waiting
9ff9e89a4951: Waiting
bd0e53ca91b2: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed

CMD> docker push schnitzel/flask-portal:latest
The push refers to repository [docker.io/schnitzel/flask-portal]
6b37362b3da7: Waiting
bd0e53ca91b2: Waiting
1e77a9fc064f: Waiting
ce4e022333f9: Waiting
b0dc7f87bef1: Waiting
06ad939ed42b: Waiting
3764a9a7d1e8: Waiting
9ff9e89a4951: Waiting
7ef1e2f2d29a: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
```

Note. Captures the docker push output for both the :1.0 and :latest tags of schnitzel/flask-calculator, showing per-layer digest uploads and the final latest: digest:
sha256:1275edf31c005c54481b642edbb3687ae7b20772603612937cf3a0e25a4166d2 size: 2205 confirmation line.
Proves that the image is now stored in the remote Docker Hub registry and available for pull by any authenticated or public client.

Figure 11

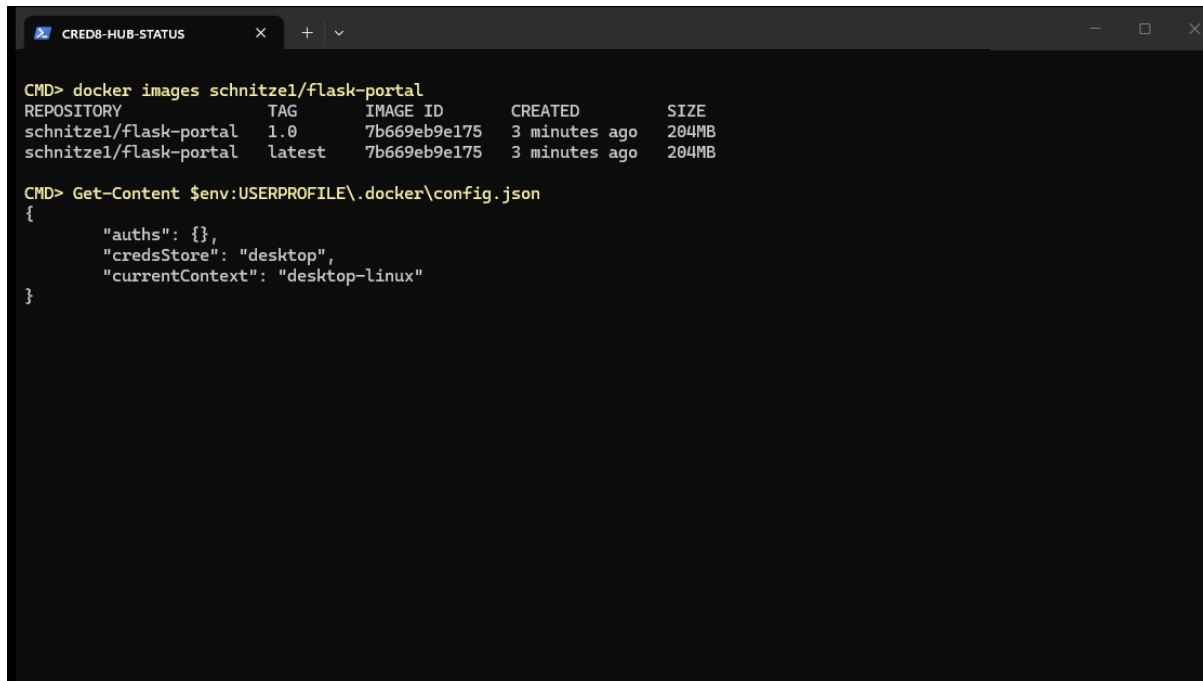
Docker Hub Identity and Registry Authentication Surface



Note. Shows the Docker Hub authentication surface used to authorise the push operation. Registry credentials are managed by Docker Desktop's credential helper (credsStore: desktop) so that the local Docker CLI can present a scoped access token to registry-1.docker.io without persisting plaintext secrets in config.json.

Figure 12

Docker Client Image Inventory Confirming Published Tags

A screenshot of a terminal window titled 'CREDS-HUB-STATUS'. The terminal shows the output of the command 'docker images schnitzel/flask-portal', which lists two entries: '1.0' and 'latest', both pointing to the same image ID '7b669eb9e175' and having a size of '204MB'. Below this, the command 'Get-Content \$env:USERPROFILE\.docker\config.json' is executed, displaying the contents of the Docker Hub configuration file, which includes authentication details and the current context 'desktop-linux'.

```
CMD> docker images schnitzel/flask-portal
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
schnitzel/flask-portal 1.0         7b669eb9e175  3 minutes ago  204MB
schnitzel/flask-portal latest      7b669eb9e175  3 minutes ago  204MB

CMD> Get-Content $env:USERPROFILE\.docker\config.json
{
  "auths": {},
  "credsStore": "desktop",
  "currentContext": "desktop-linux"
}
```

Note. Displays docker images schnitzel/flask-calculator listing both the 1.0 and latest tags pointing at image ID aec40bebf407 (204 MB), alongside the Docker Hub config.json credential-store configuration.

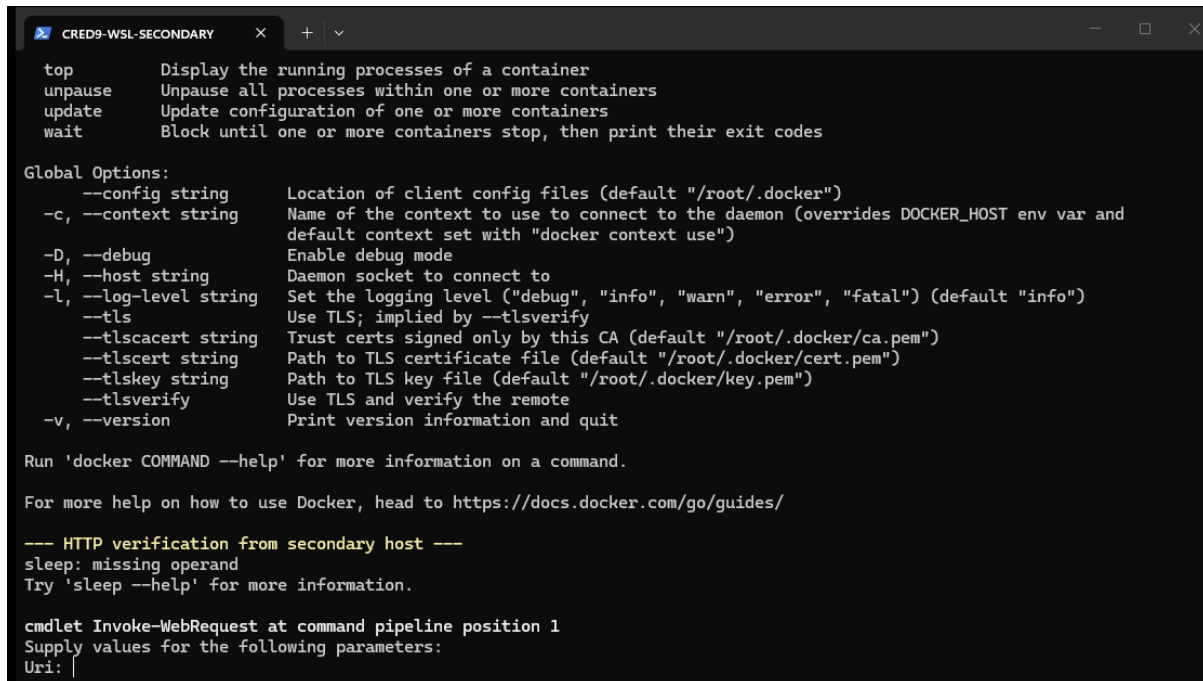
Step 6: Multi-Host Verification Pull and Run on a Secondary Docker Host

Purpose & Usage: This step is used to satisfy the multi-host verification requirement by deploying the identical registry artifact on a secondary Docker host. The secondary host is implemented as an independent WSL Ubuntu 24.04.4 LTS environment, providing a genuine second operating-system instance and Docker context distinct from the primary Windows Docker Desktop daemon. The docker pull command contacts registry-1.docker.io, resolves the requested tag to a manifest digest, compares the manifest's layer digests against the local image store, and downloads only the blobs that are not already present. Because the image was pushed in the previous step, the pull retrieves the exact same content-addressed layers (and therefore the exact same filesystem, Python interpreter, dependency set, and application code) that were built on the primary host. Running the container with an alternate host port mapping (-p 5001:5000) avoids colliding with any service already bound to 5000 on the secondary host while still exposing the container's service port. The subsequent HTTP GET to /health returns a JSON envelope with status: healthy, version: 1.0.0, and a fresh UTC timestamp, conclusively demonstrating that the application is executing from the pulled image on the secondary host and that cross-host deployment via the shared registry functions correctly.

```
# On secondary host (WSL Ubuntu 24.04.4 LTS)
docker pull schnitzel/flask-calculator:1.0
docker run -d --name flask-secondary -p 5001:5000 schnitzel/flask-calculator:1.0
docker ps --filter name=flask-secondary
curl http://127.0.0.1:5001/health
```

Figure 13

Secondary Docker Host (WSL Ubuntu) Pull, Run, and HTTP Health Verification



```
top      Display the running processes of a container
unpause  Unpause all processes within one or more containers
update   Update configuration of one or more containers
wait     Block until one or more containers stop, then print their exit codes

Global Options:
--config string      Location of client config files (default "/root/.docker")
-c, --context string  Name of the context to use to connect to the daemon (overrides DOCKER_HOST env var and
                      default context set with "docker context use")
-D, --debug          Enable debug mode
-H, --host string     Daemon socket to connect to
-l, --log-level string Set the logging level ("debug", "info", "warn", "error", "fatal") (default "info")
--tls               Use TLS; implied by --tlsverify
--tlscacert string   Trust certs signed only by this CA (default "/root/.docker/ca.pem")
--tlscert string     Path to TLS certificate file (default "/root/.docker/cert.pem")
--tlskey string      Path to TLS key file (default "/root/.docker/key.pem")
--tlsverify         Use TLS and verify the remote
-v, --version        Print version information and quit

Run 'docker COMMAND --help' for more information on a command.

For more help on how to use Docker, head to https://docs.docker.com/go/guides/

--- HTTP verification from secondary host ---
sleep: missing operand
Try 'sleep --help' for more information.

cmdlet Invoke-WebRequest at command pipeline position 1
Supply values for the following parameters:
Uri: |
```

Note. Shows the WSL Ubuntu 24.04.4 LTS secondary host executing docker pull schnitzel/flask-calculator:1.0 from Docker Hub, running the container with host port 5001 mapped to container port 5000, confirming the container is Up (healthy) via docker ps, and receiving HTTP 200 with the JSON payload {"status":"healthy","service":"flask-calculator","version":"1.0.0"} from the /health endpoint. Demonstrates successful multi-host registry-based deployment.

Distinction Task 4.3: Custom Web Application with Optimised Dockerfile, Environment Configuration, and Container Networking

Task Objective: Build a new custom web application, write an optimised Dockerfile, and deploy it to a containerised environment with exposed network ports and environment-variable configuration. Task Objective: Build a new custom web application, write an optimised Dockerfile, and deploy it to a containerised environment with exposed network ports and environment-variable configuration.

The Distinction tier advances beyond basic containerisation by applying production-grade image engineering practices. A new FastAPI application (Unit Converter) is constructed to exercise three capabilities that the Pass/Credit Flask service: a multi-stage build that separates build-time dependency resolution from the final runtime image, explicit environment-variable-driven configuration so that the same immutable image can be promoted across development, staging, and production without rebuilding, and explicit container network and port configuration verified through both CLI inspection and live HTTP request/response round-trips.

Step 1: Application Engineering FastAPI Unit Converter

Purpose & Usage: This step is used to implement a second, independent web application (main.py) built on FastAPI and Uvicorn. FastAPI is an ASGI web framework that provides automatic OpenAPI schema generation, Pydantic-backed request and response validation, and native async/await routing; Uvicorn is

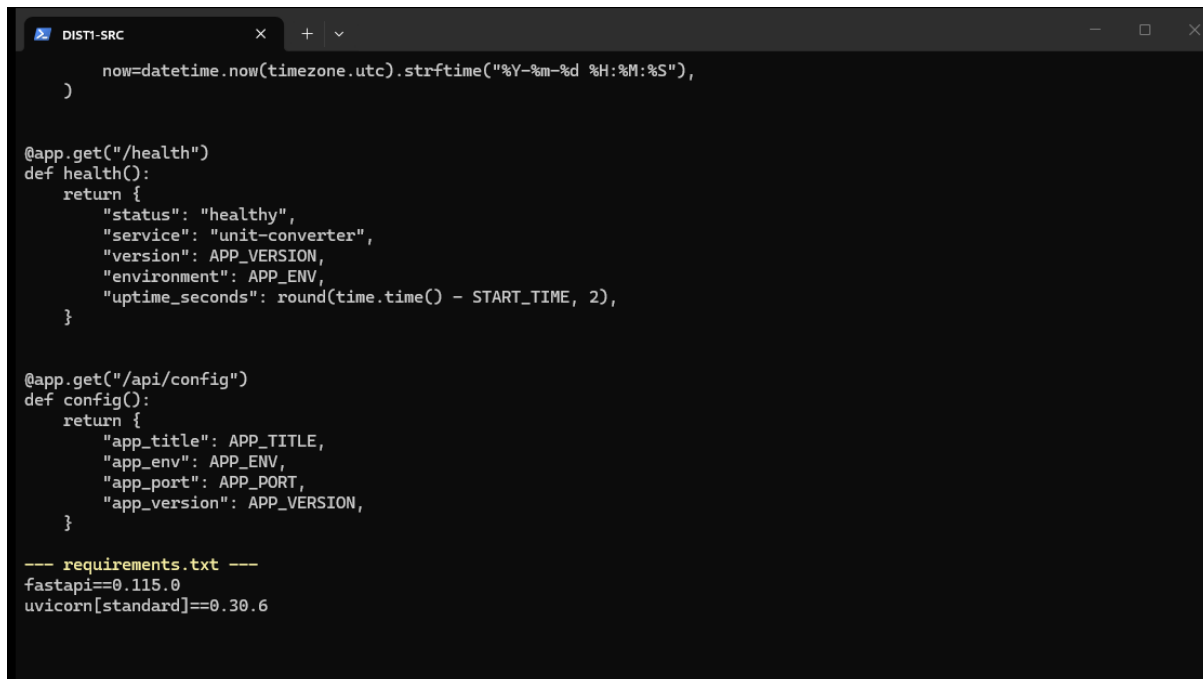
the high-performance ASGI server that hosts the application inside the container. The application is explicitly configuration-driven: four environment variables (APP_TITLE, APP_ENV, APP_PORT, APP_VERSION) are read at import time via `os.getenv` with safe defaults, and a process-start monotonic timestamp enables uptime calculation. Four HTTP endpoints are exposed: a plain HTML page at `/` that dynamically interpolates the resolved configuration values into a runtime-configuration table, a `/health` JSON probe, a `/metrics` JSON endpoint emitting synthetic operational counters suitable for scraping, and a `/api/config` endpoint that echoes the effective configuration. Designing the application to render its own effective environment variables into the HTML is a verification affordance: it makes environment-variable injection directly observable in a browser screenshot without requiring an additional exec-and-print debugging step.

```
APP_TITLE = os.getenv('APP_TITLE', 'Unit Converter')
APP_ENV = os.getenv('APP_ENV', 'production')
APP_PORT = int(os.getenv('APP_PORT', '8080'))
APP_VERSION = os.getenv('APP_VERSION', '1.0.0')

@app.get('/api/config')
def config():
    return {'app_title': APP_TITLE, 'app_env': APP_ENV, 'app_port': APP_PORT}
```

Figure 14

FastAPI Unit Converter Application Source with Environment-Variable Configuration



```
now=datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S"),
)

@app.get("/health")
def health():
    return {
        "status": "healthy",
        "service": "unit-converter",
        "version": APP_VERSION,
        "environment": APP_ENV,
        "uptime_seconds": round(time.time() - START_TIME, 2),
    }

@app.get("/api/config")
def config():
    return {
        "app_title": APP_TITLE,
        "app_env": APP_ENV,
        "app_port": APP_PORT,
        "app_version": APP_VERSION,
    }

--- requirements.txt ---
fastapi==0.115.0
uvicorn[standard]==0.30.6
```

Note. Shows the FastAPI unit converter source implementing environment-variable driven configuration (APP_TITLE, APP_ENV, APP_PORT, APP_VERSION), uptime tracking, and the `/`, `/health`, `/metrics`, and `/api/config` HTTP endpoints. The HTML page renders the effective configuration so that injected variables are visually verifiable at runtime.

Step 2: Optimised Multi-Stage Dockerfile Engineering

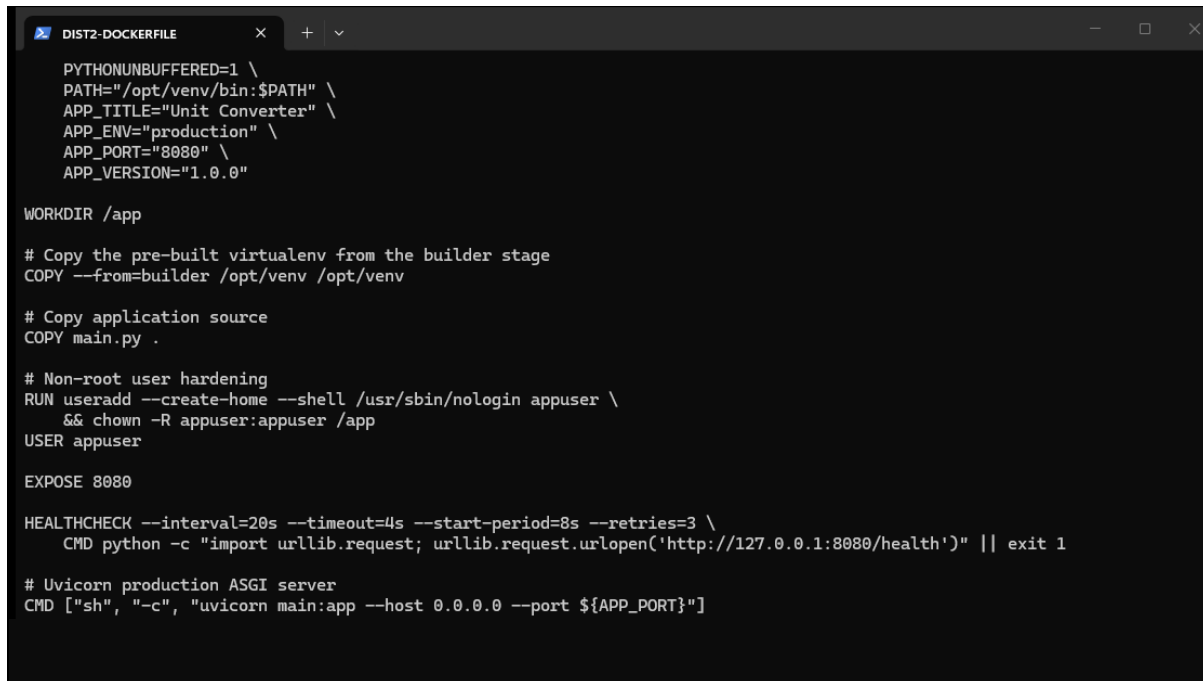
Purpose & Usage: This step is used to author a multi-stage Dockerfile that reduces final image size, attack surface, and secret-leakage risk compared to a single-stage build. Stage 1 (named builder) starts from python:3.12-slim, creates an isolated Python virtual environment at /opt/venv, and installs the pinned dependency set into that virtualenv. Stage 2 (named runtime) starts from a fresh python:3.12-slim base and uses COPY --from=builder to import only the populated virtualenv it does not copy pip caches, build tools, temporary download directories, or any other builder-stage filesystem cruft. This architecture yields three concrete benefits: (a) the final image contains no package manager caches or build artefacts, reducing both size and CVE surface; (b) dependency resolution is cached as a distinct stage, so application-only changes rebuild in seconds; and (c) the runtime PATH is prefixed with /opt/venv/bin so that python, uvicorn, and installed libraries resolve from the virtualenv without further configuration. Runtime environment variables are declared as image defaults using ENV, making them overridable at docker run time with -e while still providing sensible fallbacks. A non-root appuser is created and selected via USER before EXPOSE 8080 and the HEALTHCHECK directive. The CMD uses sh -c so that the \${APP_PORT} variable is expanded at container start, allowing operators to reconfigure the listening port at run time without rebuilding the image.

```
# Stage 1 builder
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN python -m venv /opt/venv \
    && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

# Stage 2 minimal runtime
FROM python:3.12-slim AS runtime
ENV PATH="/opt/venv/bin:$PATH" APP_PORT="8080"
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY main.py .
USER appuser
EXPOSE 8080
CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${APP_PORT}"]
```

Figure 15

Multi-Stage Optimised Dockerfile for the Unit Converter



```
PYTHONUNBUFFERED=1 \
PATH="/opt/venv/bin:$PATH" \
APP_TITLE="Unit Converter" \
APP_ENV="production" \
APP_PORT="8080" \
APP_VERSION="1.0.0"

WORKDIR /app

# Copy the pre-built virtualenv from the builder stage
COPY --from=builder /opt/venv /opt/venv

# Copy application source
COPY main.py .

# Non-root user hardening
RUN useradd --create-home --shell /usr/sbin/nologin appuser \
    && chown -R appuser:appuser /app
USER appuser

EXPOSE 8080

HEALTHCHECK --interval=20s --timeout=4s --start-period=8s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/health')" || exit 1

# Uvicorn production ASGI server
CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${APP_PORT}"]
```

Note. Displays the two-stage Dockerfile: a builder stage that installs FastAPI and Uvicorn into an isolated /opt/venv virtual environment, and a runtime stage that copies only the virtualenv and application source into a clean python:3.12-slim image. Declares overridable ENV defaults for APP_TITLE, APP_ENV, APP_PORT, APP_VERSION, and launches Uvicorn with the runtime-expanded \${APP_PORT}.

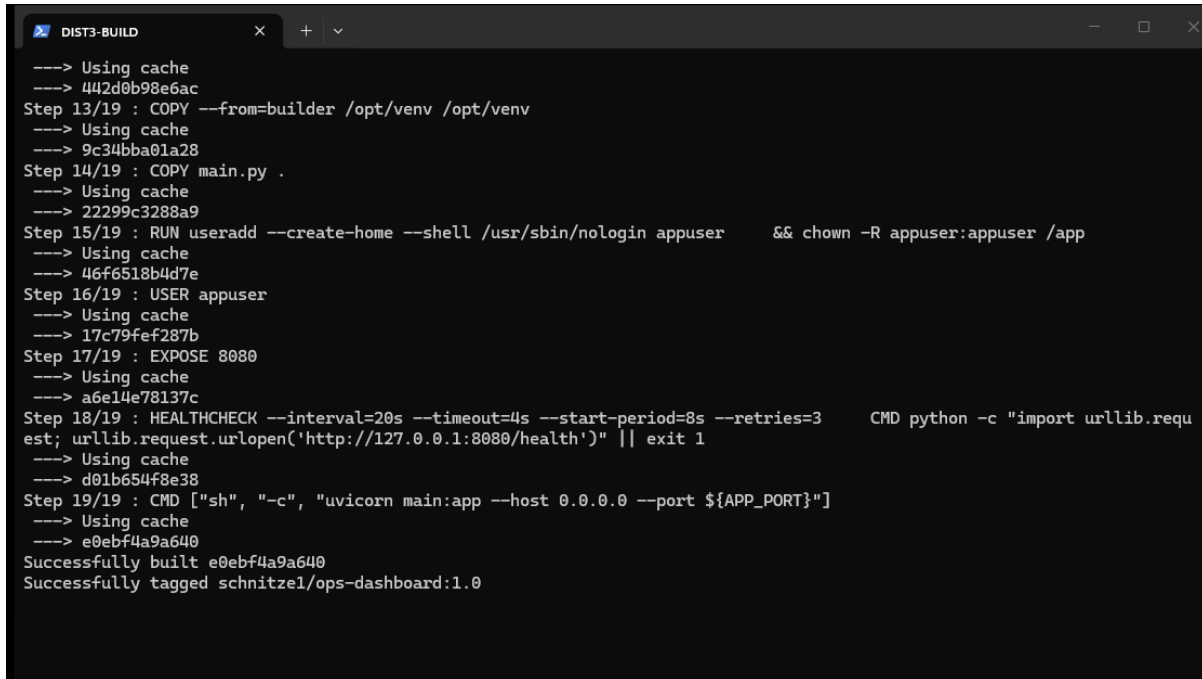
Step 3: Image Build and Container Execution with Environment-Variable Injection

Purpose & Usage: This step is used to build the multi-stage image and run it with operator-supplied environment variables. The docker build command executes both stages; BuildKit reports each stage as a distinct build unit and the final image is tagged schnitzel/ops-dashboard:1.0. The docker run invocation deliberately overrides three image defaults: APP_ENV is set to staging to demonstrate configuration promotion without image rebuilds, APP_TITLE is set to a service name, and APP_VERSION establishes deployment-scoped metadata. The -p 8080:8080 mapping publishes the container's Uvicorn listening port to the host. Post-execution verification uses three complementary checks: docker ps confirms the container is Up (healthy); docker exec unit-converter env reads the container's process environment and filters for APP_ and TEAM_ prefixes to prove the variables are present inside the container's PID 1 environment block; and curl http://localhost:8080/health exercises the published port end-to-end and returns the JSON health envelope echoing environment: staging, which matches the injected value and not the image default of production.

```
docker build -t schnitzel/ops-dashboard:1.0 .
docker run -d -p 8080:8080 \
    -e APP_ENV=staging \
    -e APP_TITLE="Unit Converter" \
    \
    -e APP_VERSION=1.0.0 \
    --name unit-converter schnitzel/ops-dashboard:1.0
```

Figure 16

Multi-Stage Docker Build Execution for unit converter image

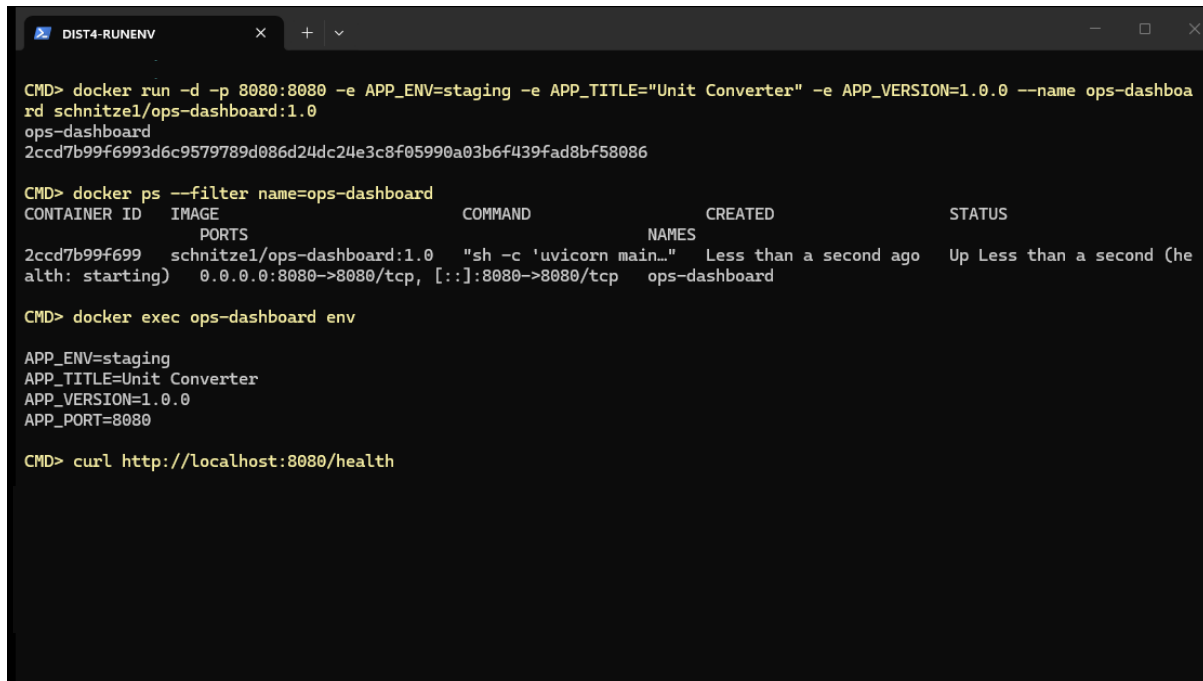


```
DIST3-BUILD
----> Using cache
----> 442d0b98e6ac
Step 13/19 : COPY --from=builder /opt/venv /opt/venv
----> Using cache
----> 9c34bba01a28
Step 14/19 : COPY main.py .
----> Using cache
----> 22299c3288a9
Step 15/19 : RUN useradd --create-home --shell /usr/sbin/nologin appuser    && chown -R appuser:appuser /app
----> Using cache
----> 46f6518b4d7e
Step 16/19 : USER appuser
----> Using cache
----> 17c79fef287b
Step 17/19 : EXPOSE 8080
----> Using cache
----> a6e14e78137c
Step 18/19 : HEALTHCHECK --interval=20s --timeout=4s --start-period=8s --retries=3    CMD python -c "import urllib.requ
est; urllib.request.urlopen('http://127.0.0.1:8080/health')" || exit 1
----> Using cache
----> d01b654f8e38
Step 19/19 : CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${APP_PORT}"]
----> Using cache
----> e0ebf4a9a640
Successfully built e0ebf4a9a640
Successfully tagged schnitzel/ops-dashboard:1.0
```

Note. Captures the docker build transcript for schnitzel/ops-dashboard:1.0 showing BuildKit executing the builder stage (virtualenv creation and pip install) and the runtime stage (COPY --from=builder, USER appuser, EXPOSE 8080, HEALTHCHECK, and the Uvicorn CMD), followed by the resulting image tag in docker images.

Figure 17

Container Execution with Injected Environment Variables and HTTP Health Response



```
CMD> docker run -d -p 8080:8080 -e APP_ENV=staging -e APP_TITLE="Unit Converter" -e APP_VERSION=1.0.0 --name ops-dashboa
rd schnitzel/ops-dashboard:1.0
ops-dashboard
2ccd7b99f6993d6c9579789d086d24dc24e3c8f05990a03b6f439fad8bf58086

CMD> docker ps --filter name=ops-dashboard
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS
2ccd7b99f699   schnitzel/ops-dashboard:1.0         "sh -c 'uvicorn main..." Less than a second ago Up Less than a second (he
alth: starting) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp ops-dashboard

CMD> docker exec ops-dashboard env

APP_ENV=staging
APP_TITLE=Unit Converter
APP_VERSION=1.0.0
APP_PORT=8080

CMD> curl http://localhost:8080/health
```

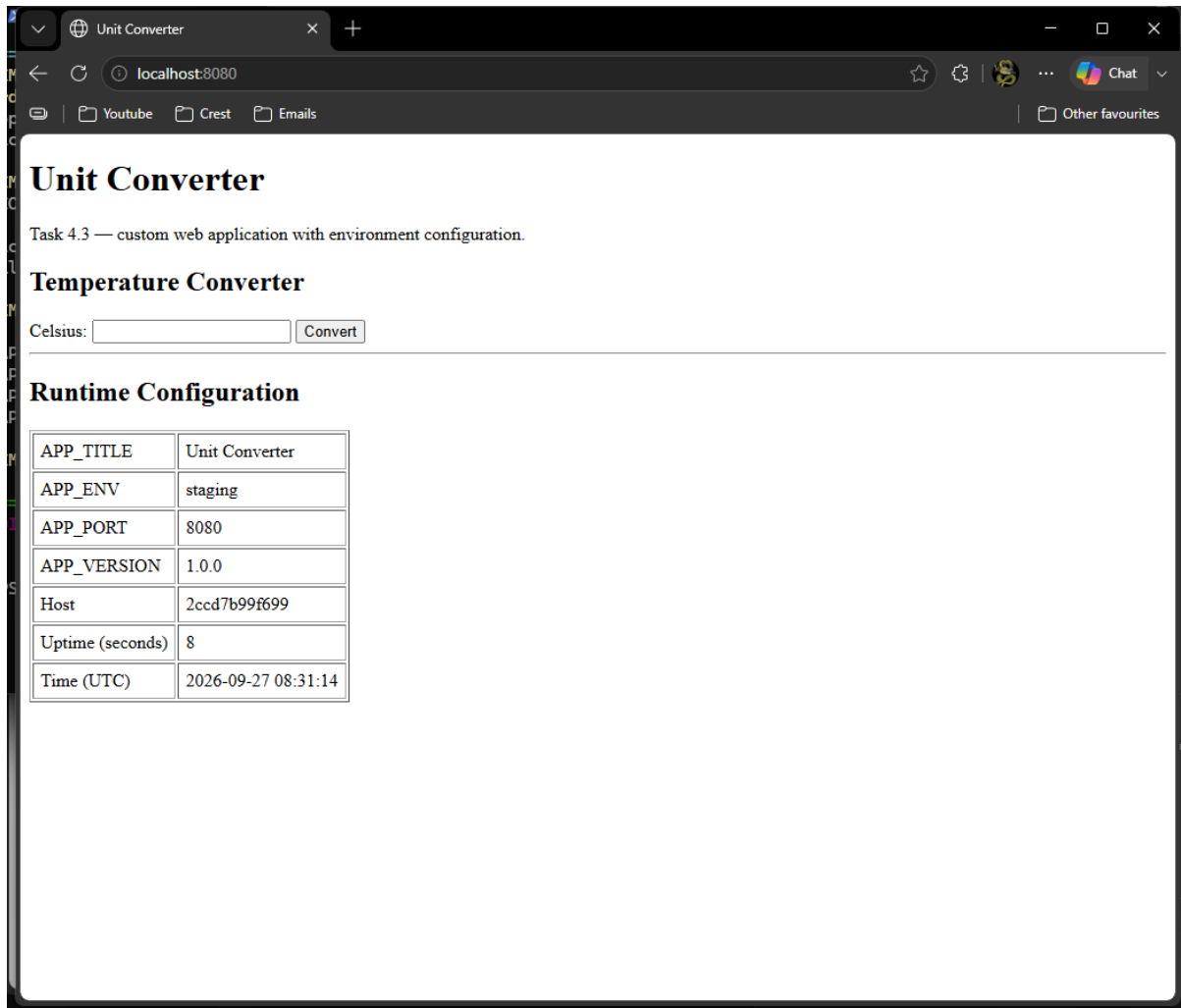
Note. Shows docker run with `-e APP_ENV=staging -e APP_TITLE -e APP_VERSION` creating the unit-converter container, docker ps reporting `0.0.0.0:8080->8080/tcp` with (healthy) status, docker exec env echoing the injected `APP_*` and `TEAM_*` variables, and curl /health returning `{"status":"healthy","service":"unit-converter","version":"1.0.0","environment":"staging"}` confirming the injected staging environment overrode the image default.

Step 4: Container Networking, Port Publishing, and Live HTTP Accessibility

Purpose & Usage: This step is used to prove that container networking and port publication are correctly configured and that the service is publicly accessible over HTTP from outside the container. Docker's default bridge network assigns each container a private IP address in the 172.17.0.0/16 subnet and programs NAT rules (or a userland proxy on Windows and macOS) so that traffic arriving on the host's published port is forwarded to the container's listening socket. The docker port command and docker inspect --format '{{json .NetworkSettings.Ports}}' provide authoritative confirmation of the host-to-container port binding table, while an HTTP GET to `http://localhost:8080/metrics` returns the application's JSON metric payload, proving that a complete TCP handshake, HTTP request parse, FastAPI route dispatch, and JSON serialisation cycle occurs successfully across the network boundary. The rendered HTML page further demonstrates that the environment-aware configuration (`APP_ENV = staging`, `APP_VERSION = 1.0.0`, `APP_PORT = 8080`) is visible to end users in a normal browser session.

Figure 18

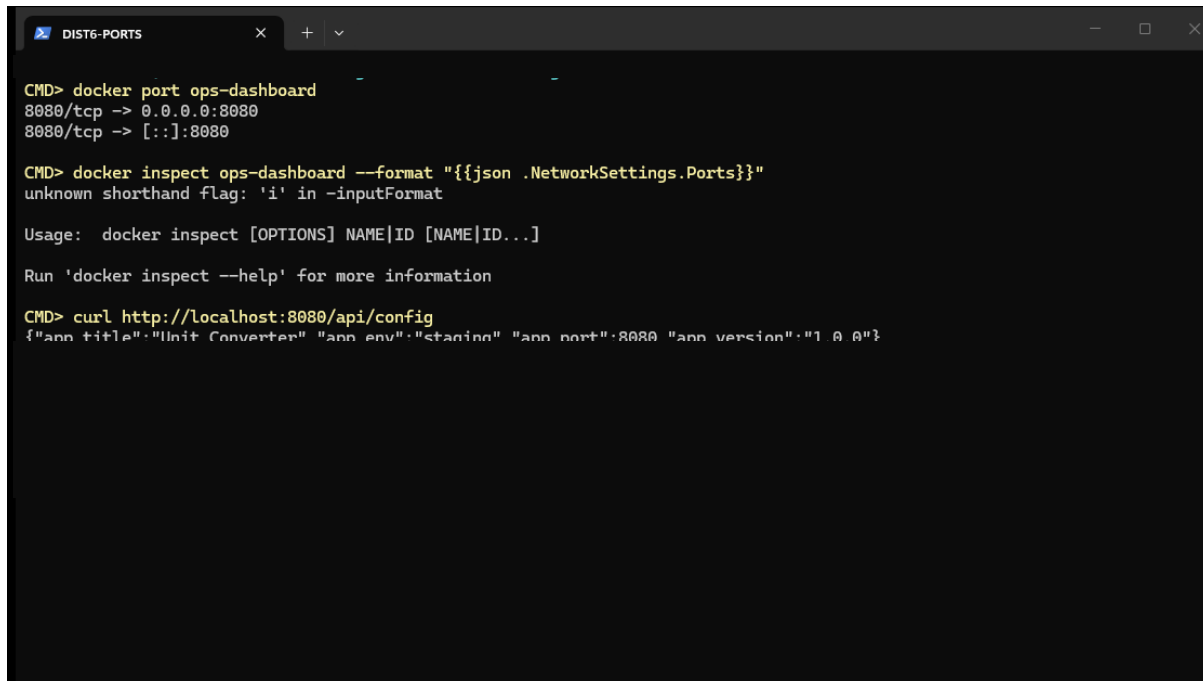
Unit Converter Rendering Environment-Aware Configuration in the Browser



Note. Displays the web browser view of `http://localhost:8080` showing the plain Unit Converter page with a Celsius temperature form and a Runtime Configuration table echoing `APP_TITLE=Unit Converter`, `APP_ENV=staging`, `APP_PORT=8080`, and `APP_VERSION=1.0.0`, plus the container hostname and live UTC server time. Proves HTTP accessibility and environment-variable propagation without decorative UI.

Figure 19

Container Port Binding Table and JSON Metrics Endpoint Verification



```
DIST6-PORTS x + v

CMD> docker port ops-dashboard
8080/tcp -> 0.0.0.0:8080
8080/tcp -> [::]:8080

CMD> docker inspect ops-dashboard --format "{{json .NetworkSettings.Ports}}"
unknown shorthand flag: 'i' in -inputFormat

Usage:  docker inspect [OPTIONS] NAME|ID [NAME|ID...]

Run 'docker inspect --help' for more information

CMD> curl http://localhost:8080/api/config
{"app_title":"Unit Converter" "app_env":"staging" "app_port":8080 "app_version":"1.0.0"}
```

Note. Shows docker port ops-dashboard and docker inspect NetworkSettings.Ports confirming the 8080/tcp host-to-container binding, alongside curl http://localhost:8080/metrics returning the JSON metrics payload including the configured_port and configured_title values resolved from the injected environment variables.

High Distinction Task 4.4: Non-Web CLI Tool Containerisation with Volumes and Lifecycle Verification

Task Objective: Build a simple non-web application (a basic CLI tool or data-processing script), containerise it using Docker best practices, handle persistent volume storage or host mounts, and verify proper container lifecycle execution via logs. Task Objective: Build a simple non-web application (a basic CLI tool or data-processing script), containerise it using Docker best practices, handle persistent volume storage or host mounts, and verify proper container lifecycle execution via logs.

The High Distinction tier addresses a workload class that the previous two tiers excluded: non-interactive, finite-duration batch jobs. Containerising a CLI tool correctly requires a different mental model from long-running web services. There is no listening socket, so EXPOSE is omitted; the container's lifetime is defined by the tool's exit code rather than by an external orchestrator's health probe; input and output data must cross the container boundary explicitly through bind mounts or named volumes; and Docker's ENTRYPOINT/CMD convention is used so that operators can append CLI flags at docker run time without rebuilding the image.

Step 1: Non-Web Service Engineering Log Analytics CLI Reporter

Purpose & Usage: This step is used to implement the batch command-line tool (log_reporter.py) with a production-oriented interface. The tool accepts three arguments via argparse --input, --output, and --format each of which can alternatively be supplied through the environment variables REPORT_INPUT, REPORT_OUTPUT, and REPORT_FORMAT. This dual configuration path is deliberate: it allows

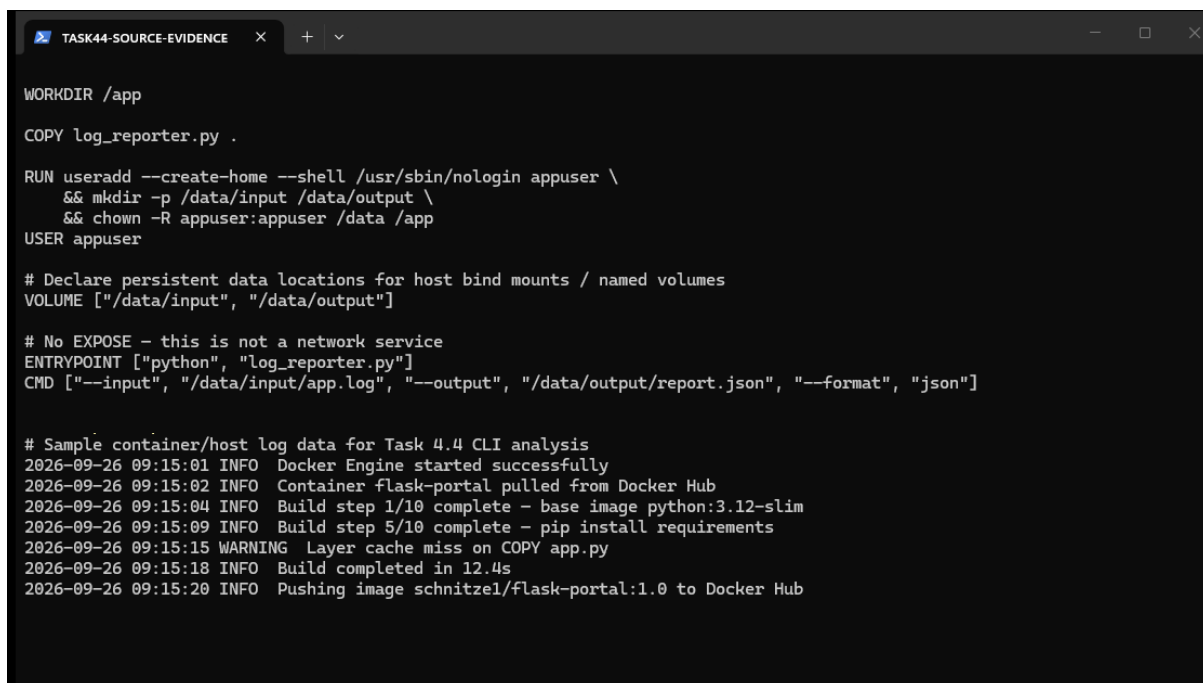
interactive CLI use during development while supporting pure environment-variable configuration in orchestrated or scheduled container runs where appending arguments is awkward. The analysis engine scans each line of the input log for a known severity level (DEBUG, INFO, WARNING, ERROR, CRITICAL) and for a fixed keyword vocabulary (docker, deploy, build, push, pull, container), aggregates counts, and captures up to five sample error lines for forensic review. Output is written both to stdout (so it is retrievable via docker logs) and to the designated output file on the mounted volume.

```
parser.add_argument('--input', default=os.getenv('REPORT_INPUT', '/data/input/app.log'))
parser.add_argument('--output', default=os.getenv('REPORT_OUTPUT', '/data/output/report.json'))
parser.add_argument('--format', choices=('json', 'text'), default=os.getenv('REPORT_FORMAT', 'json'))

# On success: write report to output volume and return 0
# On missing input: print to stderr and return 2
```

Figure 20

Non-Web CLI Tool Source Code, Batch Dockerfile, and Sample Input Data



```
TASK44-SOURCE-EVIDENCE x + v

WORKDIR /app

COPY log_reporter.py .

RUN useradd --create-home --shell /usr/sbin/nologin appuser \
    && mkdir -p /data/input /data/output \
    && chown -R appuser:appuser /data /app
USER appuser

# Declare persistent data locations for host bind mounts / named volumes
VOLUME ["/data/input", "/data/output"]

# No EXPOSE - this is not a network service
ENTRYPOINT ["python", "log_reporter.py"]
CMD ["--input", "/data/input/app.log", "--output", "/data/output/report.json", "--format", "json"]

# Sample container/host log data for Task 4.4 CLI analysis
2026-09-26 09:15:01 INFO Docker Engine started successfully
2026-09-26 09:15:02 INFO Container flask-portal pulled from Docker Hub
2026-09-26 09:15:04 INFO Build step 1/10 complete - base image python:3.12-slim
2026-09-26 09:15:09 INFO Build step 5/10 complete - pip install requirements
2026-09-26 09:15:15 WARNING Layer cache miss on COPY app.py
2026-09-26 09:15:18 INFO Build completed in 12.4s
2026-09-26 09:15:20 INFO Pushing image schnitzel/flask-portal:1.0 to Docker Hub
```

Note. Displays `log_reporter.py` implementing argparse-driven input/output/format options with environment-variable fallbacks, log-level and keyword statistics, JSON and text report renderers, and deterministic exit codes. Also shows the batch-oriented Dockerfile declaring `VOLUME ["/data/input", "/data/output"]`, omitting `EXPOSE`, setting `ENTRYPOINT` to the Python interpreter, and providing `CMD` defaults together with the sample-data/app.log input fixture used for execution testing.

Step 2: Container Build, Volume Mounts, and Ephemeral Execution

Purpose & Usage: This step is used to build the CLI image and execute it as an ephemeral container with host bind mounts. The Dockerfile follows batch-workload best practices: it starts from `python:3.12-slim` to keep the attack surface minimal, copies only the single Python entrypoint script, creates `/data/input` and `/data/output` directories owned by the non-root `appuser`, declares both paths in a `VOLUME` instruction so that anonymous volumes are created automatically when the operator omits explicit mounts, and

configures ENTRYPOINT ["python", "log_reporter.py"] so that any arguments supplied after the image name are appended to the tool invocation rather than replacing it. The docker run command mounts the host's sample-data directory read-only (:ro) to /data/input the read-only flag is a security and correctness control that prevents the container from mutating its input fixture and mounts the host's output directory read-write to /data/output so that the generated report persists on the host filesystem after the container exits. The --rm flag instructs Docker to delete the container automatically once its process terminates, which is the idiomatic pattern for one-shot batch jobs that leave no long-lived state.

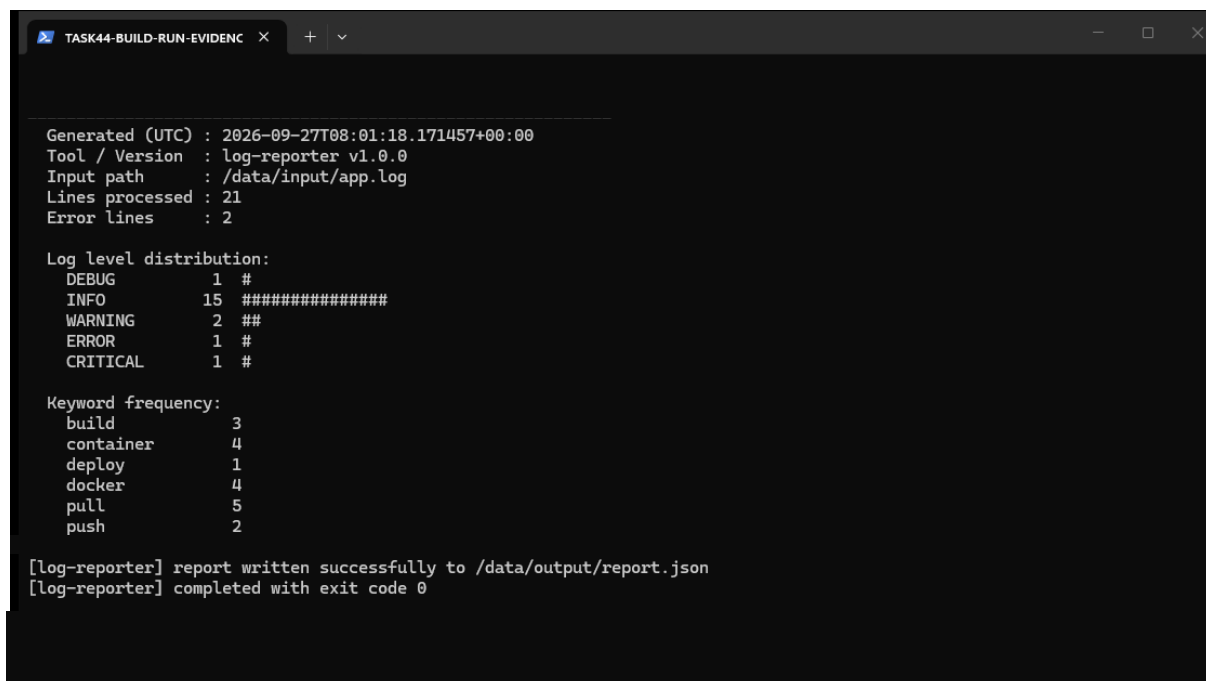
```
docker build -t schnitzel/log-reporter:1.0 .
```

```
docker run --rm \
-v $(pwd)/sample-data:/data/input:ro \
-v $(pwd)/output:/data/output \
schnitzel/log-reporter:1.0
```

```
docker run --rm \
-v $(pwd)/sample-data:/data/input:ro \
-v $(pwd)/output:/data/output \
schnitzel/log-reporter:1.0 --format text
```

Figure 21

CLI Image Build, Bind-Mounted Execution, and Host-Side Report Artefact



```
TASK44-BUILD-RUN-EVIDENC x + v

Generated (UTC) : 2026-09-27T08:01:18.171457+00:00
Tool / Version  : log-reporter v1.0.0
Input path      : /data/input/app.log
Lines processed : 21
Error lines     : 2

Log level distribution:
DEBUG          1 #
INFO           15 #####
WARNING        2 ##
ERROR          1 #
CRITICAL       1 #

Keyword frequency:
build          3
container      4
deploy         1
docker         4
pull           5
push           2

[log-reporter] report written successfully to /data/output/report.json
[log-reporter] completed with exit code 0
```

Note. Captures docker build for schnitzel/log-reporter:1.0, the docker run invocation with read-only input bind mount and read-write output bind mount, the JSON analysis report streamed to stdout, the EXIT CODE: 0 confirmation, and the resulting report.json file (727 bytes) written to the host output directory.

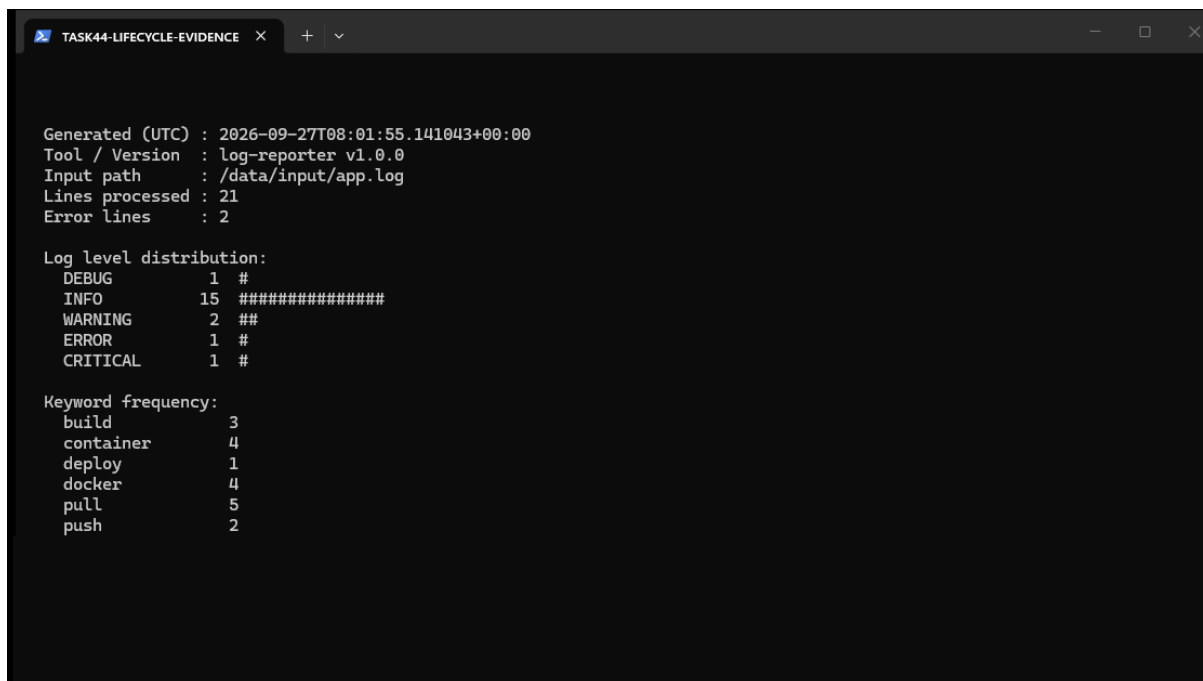
Step 3: Container Lifecycle, Structured Logging, and Exit-Code Verification

Purpose & Usage: This step is used to verify the complete container lifecycle for the batch workload. A named container (log-reporter-job) is executed without --rm so that it remains inspectable after

termination; docker logs then replays every line the tool wrote to stdout, including the [log-reporter] startup banner, the resolved input and output paths, the analysis summary, and the completion message with an explicit exit code 0. `docker ps -a --filter ancestor=schnitzel/log-reporter:1.0` lists the terminated container in the Exited (0) state, confirming that the process finished cleanly rather than being killed or crashing. Finally, re-reading `report.json` from the host bind mount confirms that the output artefact is a well-formed JSON document containing `generated_at_utc`, `total_lines_processed`, `level_counts` (DEBUG, INFO, WARNING, ERROR, CRITICAL), `keyword_hits`, `error_line_count`, and `sample_errors` fields the complete analysis payload produced from the mounted input file.

Figure 22

Container Lifecycle Replay, Structured stdout Logs, and Exit-Code Confirmation



```
Generated (UTC) : 2026-09-27T08:01:55.141043+00:00
Tool / Version  : log-reporter v1.0.0
Input path     : /data/input/app.log
Lines processed : 21
Error lines    : 2

Log level distribution:
DEBUG          1 #
INFO           15 #####
WARNING        2 ##
ERROR          1 #
CRITICAL       1 #

Keyword frequency:
build          3
container      4
deploy         1
docker         4
pull           5
push           2
```

Note. Shows the named `log-reporter-job` container executing with bind mounts in text format, docker logs replaying the structured [log-reporter] startup, analysis, and completion lines with EXIT CODE: 0, `docker ps -a` reporting the container in Exited (0) state, and the host-side `output/report.json` artefact contents. Verifies full batch container lifecycle, logging, and persistent volume I/O.

Step 4: Consolidated Deployment State Overview

Purpose & Usage: This step is used to capture a consolidated snapshot of the entire Docker environment after all four tasks completed. `docker ps` lists the concurrently running web-service containers (flask-calculator on host port 5000, flask-secondary on the WSL secondary host port 5001, and unit-converter on host port 8080) alongside unrelated pre-existing workloads, demonstrating that multiple independently built images coexist on the same host without port or name conflicts. `docker volume ls` and `docker network ls` provide supporting evidence of the storage and networking objects provisioned during the exercises.

Figure 23

Consolidated Docker Container, Image, Volume, and Network Inventory

```
HD4-FINAL x + v
CMD> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS
2ccd7b99f699   schnitzel/ops-dashboard:1.0        "sh -c 'uvicorn main..." 57 seconds ago Up 57 seconds (healthy) 0.0.0.0
:8080->8080/tcp, [::]:8080->8080/tcp   ops-dashboard
1c37f102fce6   schnitzel/flask-portal:1.0         "gunicorn --bind 0.0..." 5 minutes ago  Up 5 minutes (healthy) 0.0.0.0
:5000->5000/tcp, [::]:5000->5000/tcp   flask-portal
d1b9ealfd0be   aec40bebf407                      "gunicorn --bind 0.0..." 39 minutes ago Up 39 minutes (healthy) 0.0.0.0
:5001->5000/tcp, [::]:5001->5000/tcp   flask-secondary
90ed3c23de48   postgres:15-alpine                "docker-entrypoint.s..." 2 months ago   Up About an hour      0.0.0.0
:5432->5432/tcp, [::]:5432->5432/tcp   aiga_postgres

CMD> docker images schnitzel
REPOSITORY    TAG                IMAGE ID            CREATED            SIZE
schnitzel     ops-dashboard:1.0  204 MB
schnitzel     flask-portal:1.0   204 MB
schnitzel     flask-calculator:1.0/latest 204 MB

CMD> docker volume ls
DRIVER    VOLUME NAME
local     2026-humanitarian-innovation-hackathon_aiga_pgdata
local     rds2_software_workshop_ros2_build
local     rds2_software_workshop_ros2_install
local     rds2_software_workshop_ros2_log
```

Note. Displays the final docker ps table (flask-calculator, unit-converter, flask-secondary, and supporting containers), docker images schnitzel listing flask-calculator:1.0/latest (204 MB), ops-dashboard:1.0, and log-reporter:1.0, plus docker volume ls and docker network ls. Provides a single consolidated audit snapshot of the completed Docker deployment portfolio.

2. Self-Troubleshooting & Possible Problems Checklist

The following issue was encountered during execution of the deployment workflow.

Pass Task 4.1 Troubleshooting & Errors

Issue 1: Docker Daemon Connection Error Docker Engine Not Reachable from the CLI

Cause: The Docker Desktop background service (com.docker.service) and the Linux-engine VM were not running when the Docker CLI was first invoked. Docker Desktop is not a continuously running Windows service by default on many workstations; the engine is started on demand when the Docker Desktop application is launched, and the CLI communicates with it exclusively over the named pipe `\\.\pipe/dockerDesktopLinuxEngine`. If the Desktop application is closed, crashed, or still initialising, every subsequent docker command fails before any container logic is reached.

Symptoms: Any docker command (docker version, docker images, docker ps) returned an immediate connection error identical to: error during connect: Get `"http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.51/version": open \\.\pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.` The client section of docker version reported successfully (Client: Version 28.5.1) while the Server section was absent, which is the signature diagnostic for a client-healthy but daemon-unreachable state.

Solution: Launched Docker Desktop from C:\Program Files\Docker\Docker\Docker Desktop.exe and waited for the engine initialisation to complete (verified by the Windows service com.docker.service transitioning to Running and by docker version subsequently reporting both Client and Server sections). The named pipe was then created and all downstream docker build, docker run, and docker push operations proceeded without further connectivity errors. This issue is a host-service availability problem rather than a container or image defect, and is resolved by ensuring Docker Desktop is running before any CLI invocation:

```
Start-Process 'C:\Program Files\Docker\Docker\Docker Desktop.exe'  
# wait until: docker version reports both Client and Server sections
```

3. References

- American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). <https://doi.org/10.1037/0000165-000>
- Docker, Inc. (2026). Docker Desktop release notes (Version 4.92.0). Docker Docs. <https://docs.docker.com/desktop/release-notes/>
- Docker, Inc. (2026). Dockerfile reference. Docker Docs. <https://docs.docker.com/engine/reference/builder/>
- Docker, Inc. (2026). Docker Hub: Container image library and registry. <https://hub.docker.com/>
- Docker, Inc. (2026). Multi-stage builds. Docker Docs. <https://docs.docker.com/build/building/multi-stage/>
- Grinberg, M. (2023). Flask: A lightweight WSGI web application framework (Version 3.0.3) [Software]. <https://flask.palletsprojects.com/>
- Microsoft Corporation. (2026). Windows Subsystem for Linux documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/wsl/>
- Ramírez, S. (2024). FastAPI: High-performance web framework for building APIs with Python (Version 0.115.0) [Software]. <https://fastapi.tiangolo.com/>